Advanced Process Mining

Assignment Part I

June 14, 2026, 23:59

Prof. Dr. Wil van der Aalst

Chair of Process and Data Science
RWTH Aachen University

Introduction

This assignment part addresses the following topics:

- Process discovery,
- Conformance checking,
- Simulation of object-centric event data.

As mentioned in the lectures, you may use any software tool of your choice unless a specific tool is required in the task description. For several sub-tasks we recommend writing short scripts to automate the required steps. While doing so, keep the following points in mind:

1. The primary objective is to solve the given task correctly; the solution does not need to be a general-purpose implementation that can be applied to other problems without modification.
2. If a sub-step can be performed more easily by hand, manual execution is perfectly acceptable and may even be preferable.

For example, you are not expected to develop a full-blown object-centric event-data simulation engine.

Assignment Details

- Total number of points obtainable: 100
  - Part I: 60 points
  - Part II: 40 points
- Group size: 2–3
- Deliverables: Document containing your answers, data and models that you created, main code snippets, figures

Tools

In Question 1, you are expected to use the Process Mining Framework (ProM). Although most plugins are also available in older versions and those versions may be used as well, we recommend using version 6.15. If you encounter problems—for example, if XES files cannot be imported—we recommend trying the Docker-based version available at `https://github.com/aarkue/prom615-docker`.

Deliverables

- **What to submit**
  1. A single PDF document containing all written answers to the assignment questions, hereafter referred to as the *report*.
  2. All event log files and process models that you created or used in order to answer the questions.
  3. All figures (plots, diagrams, screenshots, etc.) referenced in the report as separate files.
  4. The source code (scripts, notebooks, or programs) you wrote for the assignment. If there is no code, you can ignore the following two requirements.
- **Purpose of the code submission**
  - The code is provided primarily to demonstrate that the work was performed independently by your group.
  - It should be sufficiently commented and structured so that a reviewer can understand the functionality of each component.
- **Execution requirements**
  - We do *not* require the code to run from start to finish without additional manual steps.
  - It is therefore acceptable if the program produces intermediate artifacts that are later used by another program.
  - Nevertheless, the code must be self-contained with respect to the parts that are essential for reproducing the results presented in the PDF.
- **Formatting and organization**
  - Submit the deliverables as a compressed archive (e.g., `.zip` or `.tar.gz`) with the following folder structure:

```
assignment_Part1_<YourGroupID>/
  report.pdf
  data/
      *.xes, *.csv, *.json, *.sqlite …  (event-log files)
  models/
      *.pnml, *.bpmn, …       (process-model files)
  figures/
      *.pdf, *.svg, *.png, …  (all figures)
  code/
       *.py, *.ipynb, …  (source code)
```

Please ensure that the report is readable, that all referenced artifacts are present in the archive, and that submitted code contains enough comments to make its purpose obvious. If you have any questions regarding the required format, feel free to ask questions in the Moodle forum.

Optional Resources

- Jupyter: `https://jupyter.org/index.html`
- PM4Py: `https://github.com/process-intelligence-solutions/pm4py`
- ProM: `https://promtools.org/`

Question:	1	2	3	Total
Points:	17	13	30	60

Question 1: Process Discovery (17 points)

(a) (12 points) Consider the following four process discovery algorithms: Inductive Miner (IM) without noise filtering, State-Based Regions Discovery (SBR Discovery) including label splitting, Integer Linear Programming Miner (ILP Miner) using causal-pair, specifically directly-follows, constraints as presented in the lecture, and efficient State Traversal Miner (eST Miner) without noise filtering.

Select two pairs of algorithms such that, together, the two pairs cover all four algorithms. For each selected pair:

- Choose a control-flow construct (e.g., activity skipping, long-term dependency, or loops) and provide an event log of any size that represents this construct such that the process models discovered by the two algorithms are different. Explicitly state the selected control-flow construct in the report. For example, an event log containing a short loop construct could be used to distinguish α-Miner from another discovery algorithm.
- Use your event log to discover process models and include the resulting models as illustrations in your report.
- Identify and explain the difference between the discovered models by describing how the chosen construct is represented in each model *(approx. 2 sentences per model)*.
- Upload the corresponding `.xes` files with your submission. If the event log is small enough, also include it in the report as a *simple case-centric event log* (comp. lecture).

To create the models, please use the following ProM plugins:

- `Mine accepting Petri net with Inductive Miner (IM)` (Noise Threshold is 0), author S.J.J. Leemans
- `Mine Transition System`, author H.M.W. Verbeek (Multiset with No limit on collection size and all activities, default for the rest of the parameters) and `Convert to Petri Net using Regions`, author B.F. van Dongen
- `Integer Linear Program (ILP)-Based Process Discovery` (Express) (default parameters), author S.J. van Zelst
- `eST Miner` (default parameters), author Lisa Mannel

Note that the individual plugin's parameters are given in parentheses.

(b) (3 points) Consider the eST Miner and its activity-ordering strategy during traversal. If **only** the activity frequencies in the event log are known, which ordering of the output transitions would you choose to maximize the number of skipped places? Briefly justify your answer *(approx. 3 sentences)*.

(c) (2 points) Consider two variants of label splitting in SBR Discovery:

1. Whenever a label needs to be split, all instances of that label that belong to the same violation type are assigned the same new label (as presented in the lecture). For x violation types (enter, exit, non-crossing), this produces x new labels.
2. Whenever a label needs to be split, all instances of that label are relabeled individually. For x transitions, this produces x new labels.

Provide a transition system with exactly two events that yields different Petri nets under these two label-splitting strategies. In your solution, include the two transition systems after label splitting as well as the resulting Petri nets.

Note that two events does not mean that there are only two edges in the transition system. It refers to two distinct edge labels.

Question 2: Conformance Checking (13 points)

In this question, we consider token-based replay and alignments.

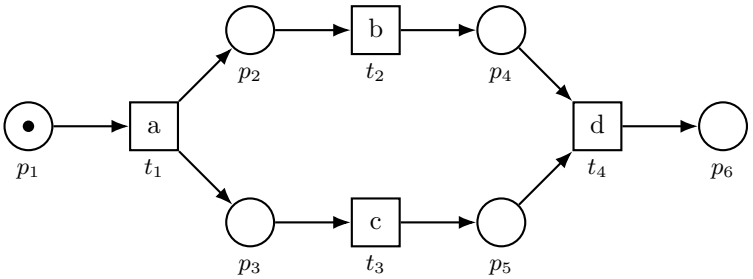
(a) (5 points) Give *one* process model and *two* traces such that token-based replay fitness is higher in one case and alignment-based trace fitness is higher in the other case. Explain your answer briefly *(approx. 2–3 sentences)*.

(b) (5 points) Give *one* trace and *two* process models such that token-based replay fitness is higher for one model and alignment-based trace fitness is higher for the other model. Explain your answer briefly *(approx. 2–3 sentences)*.

(c) (3 points) Consider the workflow net shown below, with initial marking $M_i = \{p_1\}$ and final marking $M_f = \{p_6\}$. Add one *visible* transition, together with its input and output arcs, so that:

- the resulting net is still a workflow net, and
- the optimal alignments remain unchanged for every event log L, even if L contains events labeled with the new transition.

Explain why the modified net is still a workflow net and why the optimal alignments do not change *(approx. 3–4 sentences)*.



Question 3: Object-Centric Simulation (30 points)

In this question, you generate an object-centric event log L. The log should evolve around the scenario

2036 - Olympic Games in Aachen.[1]

Assume that in 2036, the city of Aachen hosts football and equitation competitions of the Olympic Summer Games. In this scenario, your log should describe a business process in one of the following domains.

Important: The domain will be assigned to your group after the group formation deadline (Sunday, May 10th, 23:59). The assignment will be communicated via Moodle, and you'll be notified via an announcement.

Domain / Industry	Example company	Services
Public transport	Deutsche Bahn (DB), ASEAG	Handling passenger transport for athletes, officials, and spectators.
Aviation	Lufthansa	Managing flights and passenger handling for foreign delegations, media crews, and traveling fans.
Ticketing	CTS Eventim	Selling, distributing, and validating spectator tickets.
Retail / catering	REWE	Supplying food and beverages to the athletes' village, hospitality areas, and venues.
Hotel / hospitality	Some hotel, AirBnB	Accommodating visitors, including booking and check-in.
Venue operations	ALRV (CHIO Soers), Tivoli Stadium	Operating venues: scheduling sessions, coordinating ground crews and athlete logistics.

Furthermore, consider the following specification of L by the requirements R(i) - R(viii):

(i) **(Number of events)** L contains at least 100 events.

(ii) **(Number of object types)** There are at least 3 object types $ot_1, \ldots, ot_{n_1}$, $n_1 \geq 3$, that are mutually distinct, each with a non-empty set of objects.

(iii) **(Number of event types)** There are at least 5 event types $et_1, \ldots, et_{n_2}$, $n_2 \geq 5$, that are mutually distinct, each with a non-empty set of events.

(iv) **(Event-to-object relations)** Each event type relates to at least one object, and each object relates to at least one event.

(v) **(Object-to-object relations)** There are distinct object types $ot_{\text{paired1}} \neq ot_{\text{paired2}}$, and there are distinct object types $ot_{\text{parent}} \neq ot_{\text{child}}$ such that:

  (a) **(One-to-one)** Each object of type ot_{paired1} is related to exactly one object of type ot_{paired2} and vice versa.

  (b) **(One-to-many)** Each object of type ot_{parent} is related to arbitrarily many objects of type ot_{child}, and each object of type ot_{child} is related to exactly one object of type ot_{parent}.

[1] `https://olympiabewerbung.nrw/aachen`

(vi) **(Object attributes)** There is an object attribute oa and an attributed object type ot_{oa} such that each object of type ot_{oa} features a static oa-value. The domain of oa is categorical and contains at least 2 distinct labels.

(vii) **(Data-driven decisions)** There are event types et_{x_1} and et_{x_2} such that every object of the attributed type ot_{oa} from R(vi) takes part in either exactly one event of type et_{x_1}, or exactly one event of type et_{x_2}, but not both. This choice should depend on the value of oa. The choice *may* be probabilistic, i.e., given the value of oa, et_{x_1} and et_{x_2} occur with a different probability.

(viii) **(Batch Synchronization)** There are distinct event types $et_{\text{group}}, et_{\text{sync}}$ and a convergent object type ot_{sync}. Events of type et_{sync} feature multiple objects of type ot_{sync} that are all synchronized with regard to a preceding event of type et_{group}. Precisely:

  (a) **(Precedence, Synchronization)** For every event of type et_{sync}, there is a preceding event of type et_{group} (*precedence*) such that these two events relate to the same set of objects of type ot_{sync} (*synchronization*).

  (b) **(Convergence)** In general, *multiple* objects are synchronized: there exists an event of type et_{sync} that features more than one object of type ot_{sync}.

Your task is to generate such a log L in the OCEL 2.0 format. The log should adhere to the above specification, and the process described by the log should reflect service operations according to the domain assigned to your group. Yet, the process itself can be simple and does not have to be overly realistic. We only require that, given your setting, the process makes sense and uses meaningful names for event types, object types and attributes. This means there should not be abstract labels (a, b, ot_1, ot_2 etc.) and that labels are somehow semantically related.

(a) (4 points) Choose a suitable process of your assigned domain that is embedded in the given scenario *2036 - Olympic Games in Aachen*. Pick concrete names for all object types, event types, attributes, and attribute values that your log will contain. Briefly explain the meaning of each such named entity *(approx. 1 sentence per entity)*. Then, explain the intended behavior of the process on a high level, i.e., without necessarily referring to every type and intended control-flow construct *(approx. 3–4 sentences)*.

(b) (3 points) Explain the relations in the process on the type level. That is, describe which event types relate to which object types (E2O), and which object types relate to which object types (O2O). Submit your answer in tabular format following the following schemata:

E2O	ot_1	...	ot_{n_1}
et_1	...	...	...
...	...	...	...
et_{n_2}	...	...	...

O2O	ot_1	...	ot_{n_1}
ot_1	...	...	...
...	...	...	...
ot_{n_1}	...	...	...

Fill each cell where a relation exists between the two types with a brief explanation on this relation *(1 sentence)*.

(c) (3 points) Design an object-centric Petri net N_1 as a normative model for your process. Explain the intended behavior of the process by referring to the transitions, places and arcs of the net *(approx. 4–5 sentences)*. Annotate the model appropriately, and submit the annotated model as a figure along with your report.

(d) Produce an object-centric event log L that adheres to the OCEL2.0 standard, based on the requirements R(i) - R(viii) and your decisions in subtasks (a)-(c). As stated in the assignment preamble, you may use any tool, programming language, or methodological approach of your choice, including a pen-and-paper solution. Your approach does not

need to deploy the net N_1 from (c) directly, but the generated log should be consistent with it. Then, solve the following tasks regarding your resulting event log L.

Hint: To create OCELs in the required formats, we recommend using the `r4pm` *library*[2].

i. (1 point) Submit L along with your report, serialized as a *.sqlite* or *.jsonocel* file.

ii. (1 point) Show that the requirements R(i), R(ii) and R(iii) (*number of events, event types* and *object types*) are satisfied.

iii. (1 point) Show that the requirements R(v)a (*one-to-one relations*) and R(v)b (*one-to-many relations*) are satisfied.

iv. (2 points) Flatten L to the object type ot_{oa} from R(vi). Discover a fitting Petri net $N_{ot_{oa}}$ on the flattened log. Discuss how the choice between the event types et_{x_1} and et_{x_2} from R(vii) is reflected in $N_{ot_{oa}}$ *(approx. 1 sentence)*. Submit a figure of $N_{ot_{oa}}$ along with your report.

v. (2 points) Flatten L to the object type ot_{sync} from R(viii). Discover a fitting Petri net $N_{ot_{sync}}$ on the flattened log. Discuss how the *precedence* constraint between the event types et_{group} and et_{sync} from R(viii)a is reflected in $N_{ot_{sync}}$ *(approx. 1 sentence)*. Submit a figure of $N_{ot_{sync}}$ along with your report.

vi. (3 points) Illustrate that the *synchronization* requirement R(viii)a is satisfied in L. To this end, choose an appropriate visualization method (using, e.g., the *Dotted Chart* in ProM) that illustrates that each et_{sync}-event features the same objects of type ot_{sync} as some preceding et_{group}-event. Add an annotated figure to your report, and interpret the figure *(approx. 2–3 sentences)*.

(e) (3 points) Discover an object-centric Petri net N_2 on the event log L. If N_2 deviates from the normative net N_1, explain the deviations *(approx. 1–2 sentences per deviation)*. Add an annotated figure of N_2 to your report.

Note: You may use the discovered model to debug your implementation, but this debugging step should not be part of your submission. Instead, any remaining deviations between your submitted models should either point at characteristics that are not necessarily undesirable (e.g. , unplanned but allowed control-flow constructs) or be explainable by properties of the deployed discovery algorithm (e.g. , noise filters, silent transitions).

(f) Define OCPQ[3] queries that answer the following questions and report the query results on your log L. For each question, state the concrete names of the involved event types, object types, attributes and values according to your choices in subtask (a). Also, where applicable, state which thresholds you choose. Include screenshots of the executed queries.

i. (1 point) According to your entity naming regarding R(vi), pick a label c from the domain of oa. Find all objects of type ot_a whose oa-value equals c.

ii. (2 points) According to your entity naming regarding R(vi) and R(vii), extend the query from subtask (f) i as follows: Find all objects of type ot_a
- whose oa-value equals c,
- *and* that are related to exactly one event of type et_{x_1},
- *and* that are not related to any event of type et_{x_2}.

iii. (2 points) Pick a threshold $k \geq 1$. According to your entity naming regarding R(v)b, find all objects of type ot_{parent} that are related via an object-to-object relation to more than k objects of type ot_{child}.

(g) (2 points) Design one OCPQ query that addresses an interesting (business) question in the context of your process. Briefly explain this purpose *(approx. 1–2 sentences)*.

[2]For an OCEL creation example, see `https://rust4pm.aarkue.eu/docs/examples/ocel-from-csv/`

[3]`https://ocpq.aarkue.eu/`

In your explanation, refer to the concrete event types and object types used in your query. Include a screenshot of the executed query.

Total for Question 3: 30